

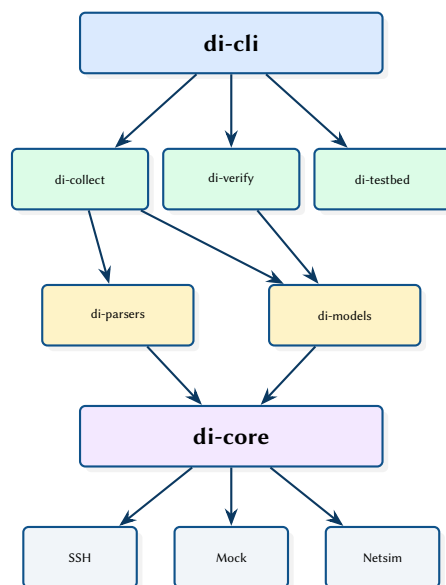
Device Interaction Framework: Type-Safe Network Device Automation in Rust

Multi-Backend Transport, Fluent Verification, and Declarative Test Suites

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. This report presents a Rust framework for network device interaction and automated testing, inspired by Cisco's pyATS but designed for type safety and performance. The framework provides multi-backend transport (SSH, Mock, Netsim), strongly-typed domain models for interfaces, routes, and ping results, a fluent verification API with domain-focused error reporting, and declarative YAML test suites that define network state assertions without code. The system is organised as seven Rust crates: core transport, parsers, models, collection, verification, testbed management, and CLI.

Contents

1	Introduction and Motivation	2
2	System Architecture	2
2.1	Transport Backends	2
3	Verification API	3
3.1	Declarative Test Suites	4
4	Design Summary	5
	List of Figures	6
	List of Tables	6
	List of Design Decisions & Rules	7
	Revision History	7

1 Introduction and Motivation

Network automation frameworks like pyATS [1] provide powerful device interaction capabilities but are constrained by Python’s runtime performance, lack of compile-time type checking, and the GIL’s impact on concurrent device access. The Device Interaction Framework reimplements these patterns in Rust, providing compile-time safety, zero-cost abstractions, and true parallelism for multi-device operations.

: Type-Safe Domain Models

All parsed device output is represented as strongly-typed Rust structs (not dictionaries or JSON). Interface status, routing table entries, and ping results are modelled with enums and newtypes, catching invalid states at compile time rather than runtime.

2 System Architecture

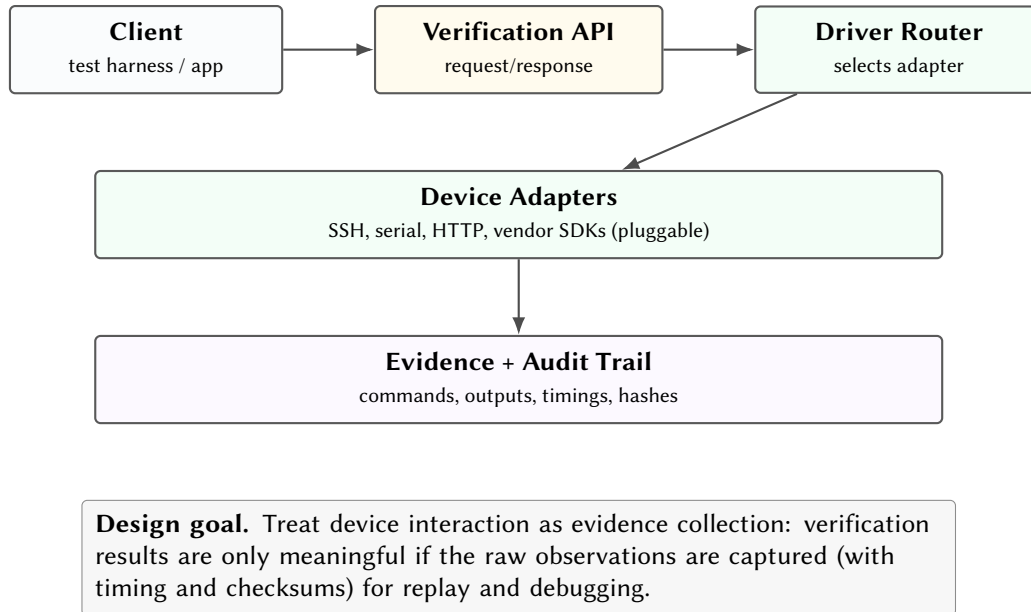


Figure 1. DeviceInteraction architecture: a stable verification API sitting above pluggable device adapters, with an explicit evidence trail for reproducibility.

The framework is organised as seven crates with a layered dependency structure:

Table 1. Crate responsibilities.

Crate	Layer	Role
di-core	Transport	Connection management, backend trait
di-parsers	Parsing	CLI output parsing (TextFSM-style)
di-models	Domain	Interface, Route, Ping models
di-collect	Collection	High-level device interaction API
di-verify	Verification	Fluent assertion engine
di-testbed	Inventory	Testbed YAML, credentials
di-cli	Application	Unified CLI binary

2.1 Transport Backends

The di-core crate defines a Transport trait that abstracts device communication:

```
#[async_trait]
pub trait Transport: Send + Sync {
    async fn connect(&mut self) -> Result<>;
    async fn execute(&mut self, command: &str) -> Result<String>;
    async fn disconnect(&mut self) -> Result<>;
    fn is_connected(&self) -> bool;
}
```

Three backends implement this trait:

- **SSH** — Production backend using `russh` for async SSH2 connections with prompt detection and command timing.
- **Mock** — Development/CI backend that replays pre-recorded command-response pairs from YAML fixtures.
- **Netsim** — Integration backend that connects to a network simulator's management API.

Design Decision 1: Mock Backend for CI

The Mock transport enables full integration testing without network hardware. Test fixtures capture real device output once, then replay deterministically. This allows the verification suite to run in CI with sub-second execution times.

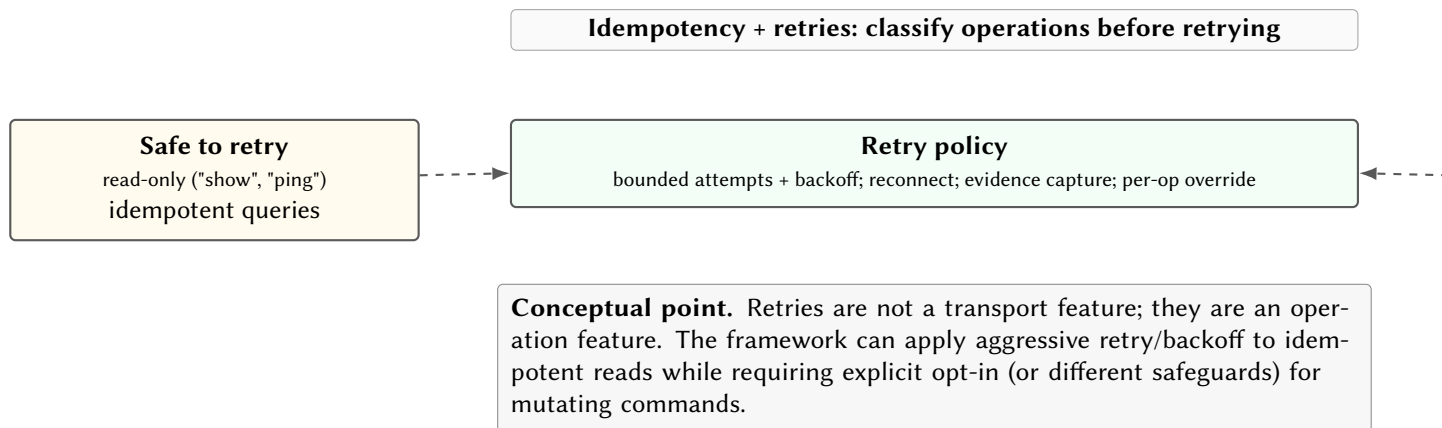


Figure 2. Retry safety depends on idempotency. Operations are classified so retry/backoff policies can be applied predictably and safely.

3 Verification API

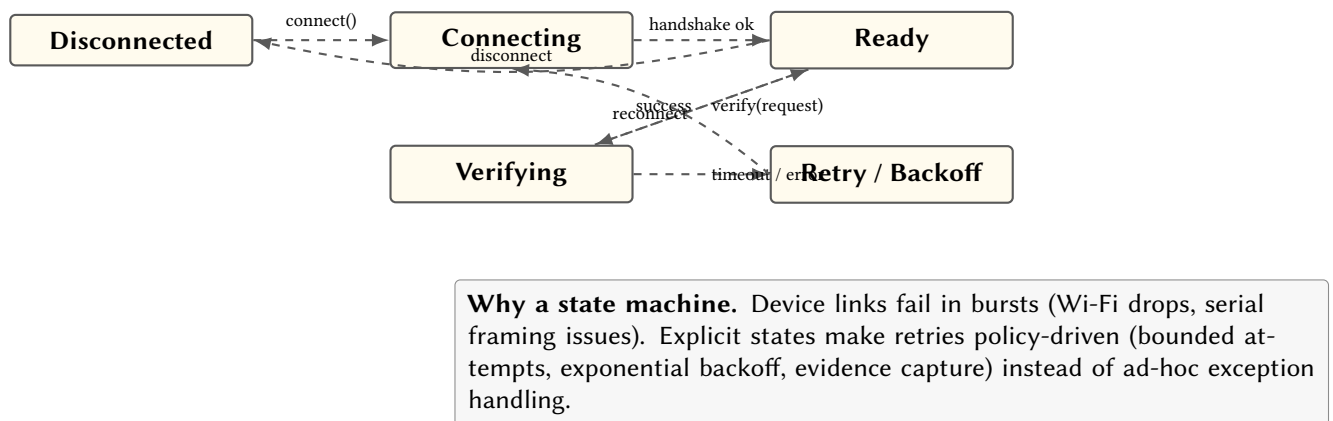


Figure 3. Interaction lifecycle as a state machine. Verification is modeled as a transient state so timeouts and retries are handled consistently across adapters.

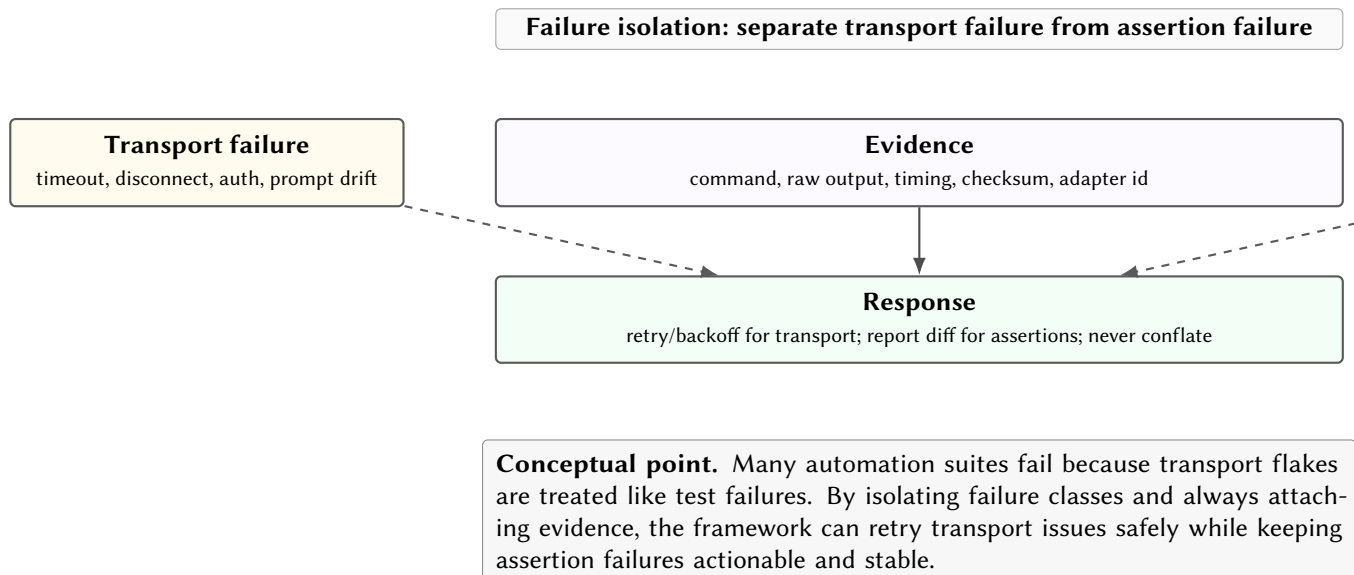


Figure 4. Failure isolation model. Transport failures trigger policy-driven retries; assertion failures produce domain-focused diffs, both grounded by the same evidence trail.

The di-verify crate provides a fluent assertion API inspired by property-based testing frameworks:

```

verify(&device)
  .interfaces()
  .named("GigabitEthernet0/0")
  .is_up()
  .has_ip("10.0.0.1/30")
  .and()
  .routes()
  .to("192.168.0.0/16")
  .via("10.0.0.2")
  .exists()
  .run()
  .await?;
  
```

Failed assertions produce domain-specific error messages:

```

FAIL: Interface GigabitEthernet0/0
  Expected: status = Up
  Actual:   status = Down (administratively)
  Device:   spine-1 (10.1.1.1)
  
```

3.1 Declarative Test Suites

For code-free testing, di-verify supports YAML test suites:

```

suite: connectivity-check
tests:
  - name: spine uplinks are up
    device: spine-1
    checks:
      - type: interface
        name: "Ethernet1"
        status: up
      - type: route
        prefix: "0.0.0.0/0"
        next_hop: "10.0.0.1"
  
```

Insight:
The YAML suite format enables network operators who do not write Rust to define automated checks. The CLI loads and executes suites against the testbed inventory, reporting results in JUnit XML format for CI integration.

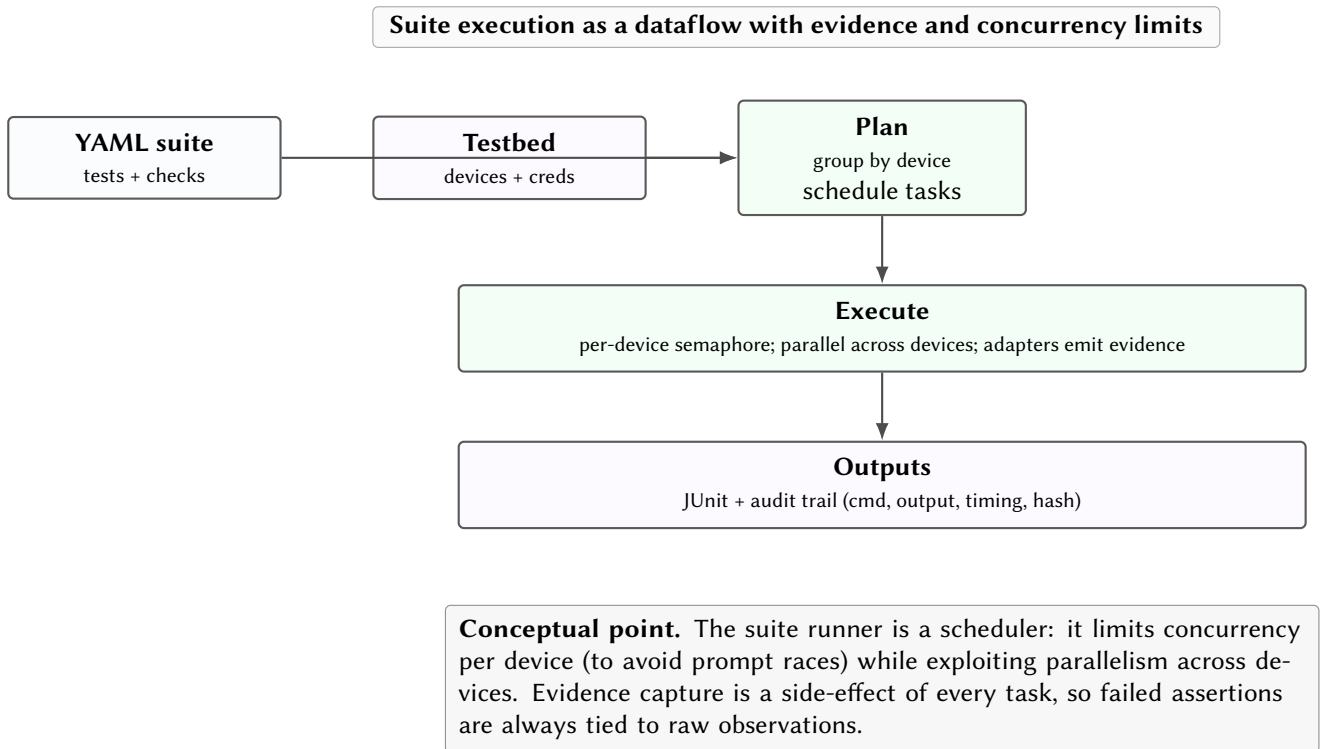


Figure 5. Declarative suites compile into scheduled tasks. Concurrency limits preserve device-session correctness while still enabling parallel multi-device execution.

4 Design Summary

Table 2. Key design decisions.

Decision	Rationale
Seven-crate workspace	Clean dependency boundaries; transport layer does not depend on domain models
Async transport trait	Enables concurrent multi-device operations without thread-per-connection
Mock backend	Deterministic CI testing without hardware; sub-second test execution
Fluent verification API	Readable assertions with domain-specific error messages
YAML test suites	Code-free testing for network operators

References

- [1] Cisco, *pyATS: Python automated test systems*, 2024. [Online]. Available: <https://developer.cisco.com/pyats/>

List of Figures

1 DeviceInteraction architecture: a stable verification API sitting above pluggable device adapters, with an explicit evidence trail for reproducibility.	2
2 Retry safety depends on idempotency. Operations are classified so retry/backoff policies can be applied predictably and safely.	3
3 Interaction lifecycle as a state machine. Verification is modeled as a transient state so timeouts and retries are handled consistently across adapters.	3
4 Failure isolation model. Transport failures trigger policy-driven retries; assertion failures produce domain-focused diffs, both grounded by the same evidence trail.	4
5 Declarative suites compile into scheduled tasks. Concurrency limits preserve device-session correctness while still enabling parallel multi-device execution.	5

List of Tables

1 Crate responsibilities.	2
2 Key design decisions.	5

List of Design Decisions & Rules

1	Design Rule: Type-Safe Domain Models	2
1	Mock Backend for CI	3

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report